



RouteZone

Application de prédiction de la gravité des accidents de la route

Rapport E1 — Collecte, stockage et mise à disposition des données

Meriem ABDELOUAHED

Formation Développeur en Intelligence Artificielle — Simplon × Microsoft

Titre RNCP37827

github.com/Meriem69/Routezone

Sommaire

1. Contexte et objectif	3
2. Collecte des données	3
3. Nettoyage et préparation	5
4. Base de données PostgreSQL	5
5. API REST Data — FastAPI	6
6. Conclusion	8
Annexe. Figures complémentaires	9

1. Contexte et objectif

RouteZone est un système de machine learning visant à prédire la gravité d'un accident routier — GRAVE (tué ou hospitalisé) ou PAS GRAVE (indemne ou blessé léger) — à partir des données BAAC (Bulletin d'Analyse des Accidents Corporels) 2022-2024, publiées en open data par le Ministère de l'Intérieur.

Le projet répond à une problématique de sécurité routière : aider les autorités et les services de secours à anticiper les cas critiques pour mieux prioriser les interventions. La spécificité de RouteZone réside dans l'intégration du Golden Hour, principe de traumatologie selon lequel une prise en charge dans les 60 minutes augmente fortement les chances de survie.

Périmètre du Bloc E1 et cheminement

Le Bloc E1 couvre la collecte, le stockage et la mise à disposition des données. Le travail s'est construit à travers quatre notebooks : notebook_01 (exploration et nettoyage), notebook_02 (enrichissement météo), notebook_03 (scraping ONISR) et notebook_04 (calcul des temps d'intervention OSRM).

Compétence	Action réalisée	Livrable
C1 — Automatiser l'extraction	12 CSV BAAC + API Open-Meteo + scraping ONISR + enrichissement OSRM	4 notebooks + 3 scripts .py
C2 — Sources fiables	Identification et validation des sources open data officielles	Documentation README
C3 — Nettoyage / transformation	Gestion des NaN, des types, des colonnes inutiles, merge des 4 tables	notebook_01 + dataset_clean.csv
C4 — Base de données	Modélisation Merise (MCD/MPD), PostgreSQL 16, scripts d'import	schema_merise.md + create_db.sql
C5 — API REST Data	Développement FastAPI, endpoints documentés sous Swagger	src/api/routes_data.py

2. Collecte des données

La collecte s'appuie sur quatre sources complémentaires : le jeu principal BAAC, deux enrichissements externes (Open-Meteo et ONISR) et un enrichissement calculé via OSRM.

Sources des données

- BAAC (accidents corporels) : data.gouv.fr — ONISR / Ministère de l'Intérieur.
- Météo : API Open-Meteo — open-meteo.com (gratuite, sans clé).
- Baromètre mensuel : ONISR — onisr.securite-routiere.gouv.fr.
- Casernes (CIS) et SAU : data.gouv.fr ; réseau routier : OpenStreetMap via OSRM.

2.1 Données BAAC — notebook_01

Les fichiers BAAC sont organisés en 4 tables liées par la clé Num_Acc : caract (caractéristiques de l'accident), lieux (route, vitesse, surface), usagers (âge, sexe, gravité, équipement) et vehicules (catégorie, type de collision). Au total, 12 fichiers CSV ont été chargés, représentant 153 054 accidents sur 2022-2024, impliquant 413 570 usagers et environ 270 000 véhicules.

```
=== caract 2024 ===
```

	Num_Acc	jour	mois	an	hrmn	lum	dep	com	agg	int	atm	col	adr	lat	long
0	2024000000001	25	3	2024	07:40	2	70	70285	1	1	5	1	D438	47,56277000	6,75832000
1	2024000000002	20	3	2024	15:05	1	21	21054	2	3	7	6	HOTEL DIEU (RUE DE L')	47,02109000	4,83755000

```
=== lieux 2022 ===
```

	Num_Acc	catr	voie	v1	v2	circ	nbv	vosp	prof	pr	pr1	plan	lartpc	larout	surf	infra	situ	vma
0	2022000000001	4	TEIL(vieille route du)	0	NaN	2	2	0	1	(1)	(1)	1	NaN	-1	1	0	1	50
1	2022000000002	4	NaN	0	NaN	2	2	0	1	(1)	(1)	1	NaN	-1	1	0	1	50

Aperçu des tables BAAC brutes caract et lieux (notebook_01)

2.2 Enrichissement Open-Meteo — notebook_02

La colonne atm du BAAC code la météo de façon déclarative et subjective. L'API Open-Meteo (gratuite, sans clé) fournit la météo réelle heure par heure par coordonnées GPS. L'enrichissement ajoute 4 variables (temperature, precipitation, windspeed, weathercode) via une boucle Python avec une pause de 0,1 s entre requêtes pour respecter l'API. La comparaison entre code déclaré et mesure réelle confirme l'intérêt de l'enrichissement.

[À insérer] capture de ton code d'appel à l'API Open-Meteo (la requête + la boucle) dans notebook_02.

2.3 Scraping ONISR — notebook_03

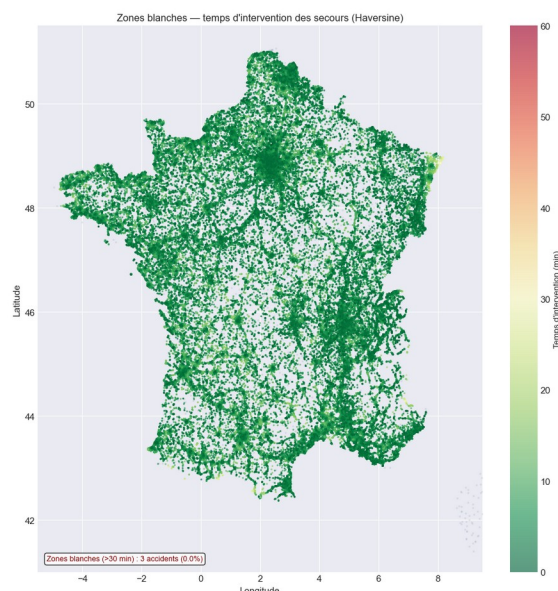
Le baromètre mensuel ONISR (nombre de tués par mois) n'est pas disponible en CSV : il est scrapé avec requests et BeautifulSoup. En inspectant le HTML, j'ai repéré le paramètre d'URL field_annees_target_id qui permet de cibler une année directement, ce qui réduit le nombre de requêtes. Ces données servent ensuite à valider la qualité du dataset BAAC par comparaison des comptages mensuels.

[À insérer] capture de ton code de scraping (requests + BeautifulSoup, repérage du paramètre field_annees_target_id) dans notebook_03.

2.4 Enrichissement OSRM — Golden Hour — notebook_04

Le BAAC ne contient aucune information sur le temps d'intervention des secours. J'ai donc intégré cette dimension comme variable explicative. OSRM (Open Source Routing Machine) calcule des itinéraires réels sur le réseau OpenStreetMap, plutôt qu'une distance à vol d'oiseau (Haversine). Le pipeline calcule, pour chaque accident, le trajet depuis la caserne de pompiers la plus proche puis vers le Service d'Accueil des Urgences (SAU), à partir de 5 906 casernes (CIS) et 638 SAU issus de data.gouv.fr. Cette feature deviendra le facteur le plus discriminant du modèle final.

[À insérer] capture de ton code OSRM (calcul du trajet) ou un extrait de la table temps_intervention remplie, dans notebook_04.



Temps d'intervention des secours cartographié par accident (notebook_04)

3. Nettoyage et préparation

Le nettoyage suit un principe simple : nettoyer chaque table indépendamment, puis fusionner, pour limiter la propagation d'erreurs. Les NaN ont d'abord été visualisés avec missingno (voir Annexe) pour décider, colonne par colonne, entre suppression et imputation.

3.1 Étapes principales

- Correction du nom de colonne Accident_Id en Num_Acc sur caract 2022 (bug qui aurait cassé les merges).
- Concaténation des 3 années par table ; remplacement des -1 par NaN (convention BAAC : -1 = non renseigné).
- Suppression des doublons stricts ; conversion des types lus comme texte ; correction des coordonnées GPS (virgule en point) et filtrage des DOM-TOM (9 578 lignes retirées).
- Suppression des colonnes à plus de 90 % de NaN ; imputation par le mode (catégorielles) et la médiane (numériques) ; suppression des lignes critiques manquantes (grav, sexe, dep).
- Vérification finale : 0 NaN. Fusion des 4 tables par left join sur Num_Acc. Création de 3 variables dérivées : age, heure, jour_semaine.

3.2 Variable cible et déséquilibre

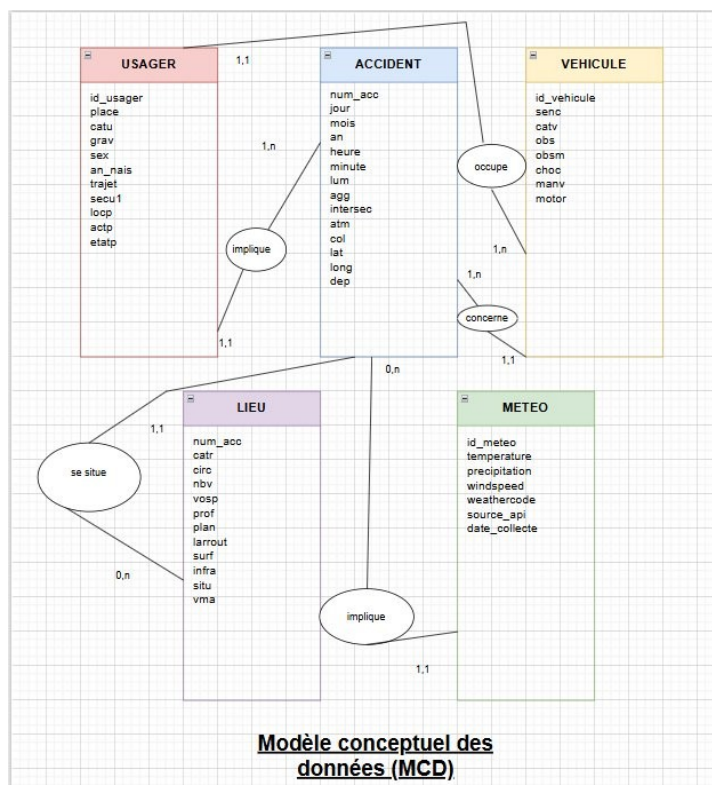
La colonne grav (1 = indemne, 2 = tué, 3 = hospitalisé, 4 = blessé léger) est binarisée : GRAVE = {2, 3}, PAS GRAVE = {1, 4}. La répartition fait apparaître environ 17 % de cas GRAVE — un déséquilibre des classes décisif pour la modélisation.

3.3 Analyses de biais

Deux analyses de biais assurent une lecture critique : biais de représentation par sexe et biais géographique (volume brut vs taux pour 100 000 habitants, voir Annexe). Elles montrent qu'un volume élevé n'est pas synonyme de risque élevé sans normalisation.

4. Base de données PostgreSQL

La base est modélisée selon la méthode Merise (MCD puis MPD) et implémentée en PostgreSQL 16, choisi pour sa conformité ACID, son support des types géospatiaux, sa gestion de la concurrence (API, scripts d'import et monitoring simultanés) et son intégration avec Docker et SQLAlchemy.



Modèle Conceptuel des Données — méthode Merise (schema_merise.md)

La base contient 9 tables en deux groupes : données métier (accidents, lieux, usagers, vehicules, meteo, barometre_onisr, temps_intervention) et données applicatives (users avec mot de passe hashé en bcrypt, predictions avec historique daté). Le script `create_db.sql` est exécuté automatiquement au démarrage du conteneur PostgreSQL via `docker-entrypoint-initdb.d`, avec des index sur les colonnes les plus requêtées. Les scripts d'alimentation sont `import_data.py` (153 054 accidents), `collect_meteo.py` et `scraping_onisr.py`.

[À insérer] capture de tes tables dans pgAdmin ou psql (commande `\dt`, ou un `SELECT COUNT(*)`) prouvant que la base est bien peuplée.

5. API REST Data — FastAPI

L'API expose les données via des endpoints REST documentés automatiquement par Swagger UI. Elle est construite avec FastAPI (basé sur Pydantic). En développement, elle est lancée par `python -m uvicorn main:app --reload` (rechargement automatique du code) ; en production, via Docker, par `uvicorn main:app --host 0.0.0.0 --port 8000` (commande définie dans le Dockerfile). L'accès aux données se fait via SQLAlchemy 2.0, qui sépare la logique SQL du code Python et protège contre les injections par le paramétrage des requêtes. L'accès est protégé par clé d'API (X-API-Key) et/ou JWT, avec validation Pydantic et documentation interactive sur `/docs` et `/redoc`.

Endpoint	Description
GET /accidents	Liste filtrable des accidents (département, année, gravité)
GET /accidents/stats	Statistiques globales (total, répartition, top départements)
GET /accidents/departement/{dep}	Accidents par département
GET /accidents/gravite	Répartition des usagers par gravité
GET /meteo/stats	Statistiques météo agrégées (Open-Meteo)
GET /onisr/barometre	Baromètre mensuel des tués (ONISR)

RouteZone API

[/openapi.json](#)

API REST du projet RouteZone -- prediction de gravite des accidents routiers. Certification RNCP37827 Dev IA Simphon x Microsoft.

Domaines :

- /accidents , /meteo , /onisr -- donnees BAAC (C5)
- /predict , /predictions -- intelligence artificielle (C8-C9)
- /auth -- authentication JWT

Authentification : header `X-API-Key` et/ou `Authorization: Bearer <JWT>`

Authorize

Auth ^

POST	/auth/register	Register	▼
POST	/auth/login	Login	▼
POST	/auth/token	Token OAuth2	▼
GET	/auth/me	Me	▼

Donnees BAAC ^

GET	/accidents	Liste Accidents	▼
GET	/accidents/stats	Statistiques Globales	▼
GET	/accidents/departement/{dep}	Accidents Par Departement	▼
GET	/accidents/gravite	Repartition Gravite	▼
GET	/meteo/stats	Stats Meteo	▼
GET	/onisr/barometre	Barometre Onisr	▼

Documentation interactive Swagger UI exposée sur /docs

[À insérer] capture d'une requête GET /accidents renvoyant du JSON avec un code 200 (depuis Swagger ou un navigateur) — preuve que l'API sert bien les données.

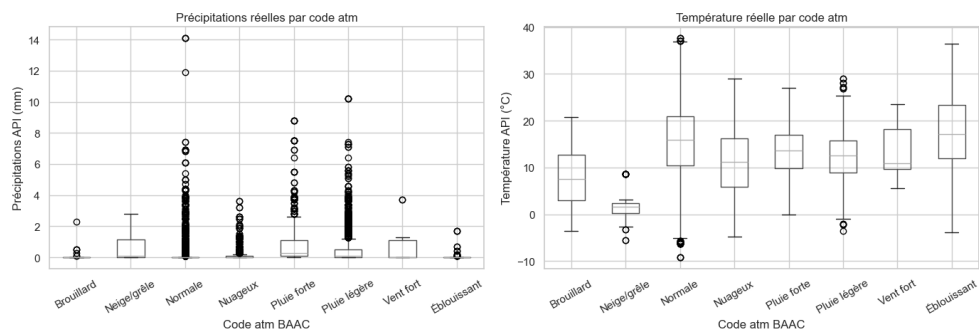
6. Conclusion

Le Bloc E1 constitue le socle de données du projet. Livrables produits : dataset nettoyé (153 054 accidents, 413 570 usagers, 0 NaN), dataset enrichi (météo + temps_total_osrm), base PostgreSQL 16 (9 tables, schéma Merise), 4 notebooks documentés et 4 scripts de production, et une API FastAPI documentée sous Swagger.

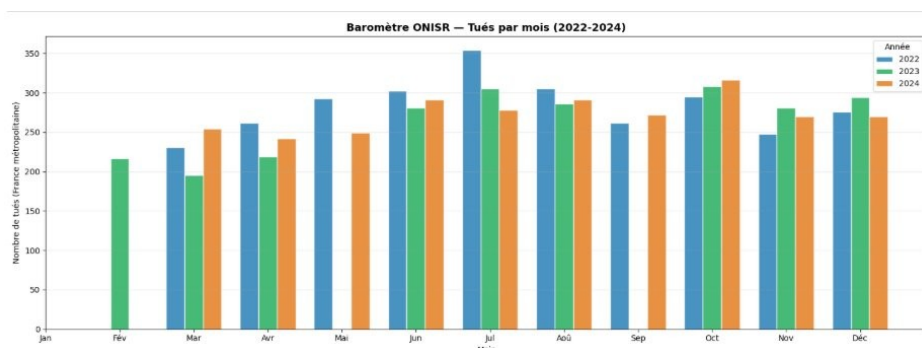
L'apport différenciant de ce bloc est l'intégration du Golden Hour via OSRM : cette feature dérivée deviendra le facteur le plus discriminant du modèle final. Le dataset enrichi et l'API alimentent le Bloc E3 : entraînement et comparaison de modèles de classification.

Annexe — Figures complémentaires

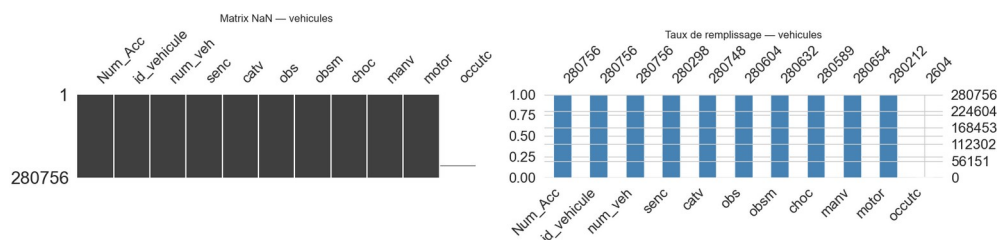
Figures de cadrage et illustrations détaillées, en complément du corps du rapport.



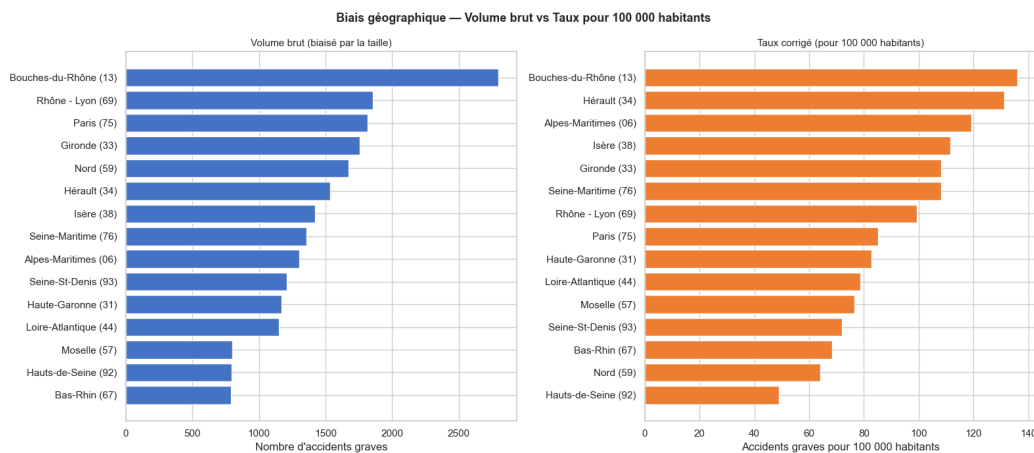
A1 — Météo réelle Open-Meteo selon le code atm déclaré du BAAC (notebook_02)



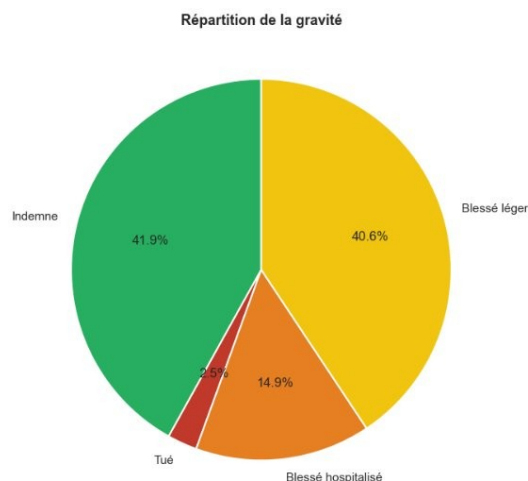
A2 — Baromètre mensuel des tués scrapé sur l'ONISR (notebook_03)



A3 — Valeurs manquantes visualisées avec missingno (notebook_01)



A4 — Biais géographique : volume brut vs taux corrigé par habitant (notebook_01)



A5 — Répartition de la variable cible grav (notebook_01)

```

caract2022 : 55,302 lignes, 15 colonnes
caract2023 : 54,822 lignes, 15 colonnes
caract2024 : 54,402 lignes, 15 colonnes

lieux2022 : 55,302 lignes, 18 colonnes
lieux2023 : 70,860 lignes, 18 colonnes
lieux2024 : 70,248 lignes, 18 colonnes

usagers2022 : 126,662 lignes, 16 colonnes
usagers2023 : 125,789 lignes, 16 colonnes
usagers2024 : 125,187 lignes, 16 colonnes

vehicules2022 : 94,493 lignes, 11 colonnes
vehicules2023 : 93,585 lignes, 11 colonnes
vehicules2024 : 92,678 lignes, 11 colonnes

```

A6 — Volumes des fichiers BAAC bruts par année

```

=== usagers 2024 ===

```

	Num_Acc	id_usager	id_vehicule	num_veh	place	catu	grav	sexe	an_nais	trajet	secu1	secu2	secu3	locp	actp	etatp
0	202400000001	203 988 581	155 781 758	A01	1	1	3	1	2003.0	2	1	-1	-1	-1	-1	-1
1	202400000001	203 988 582	155 781 759	B01	1	1	1	1	1997.0	4	1	-1	-1	-1	-1	-1

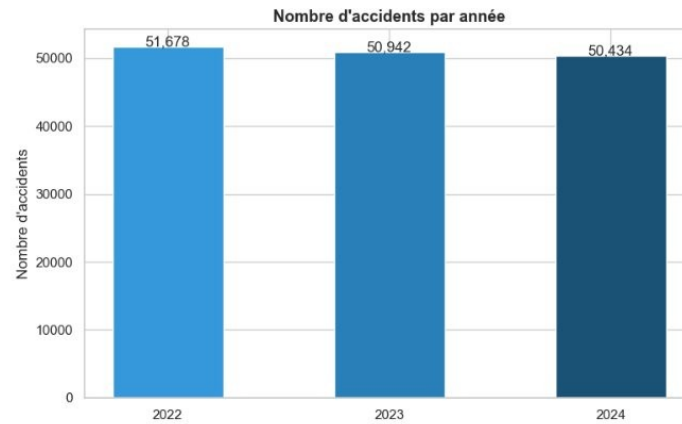
```

=== vehicules 2022 ===

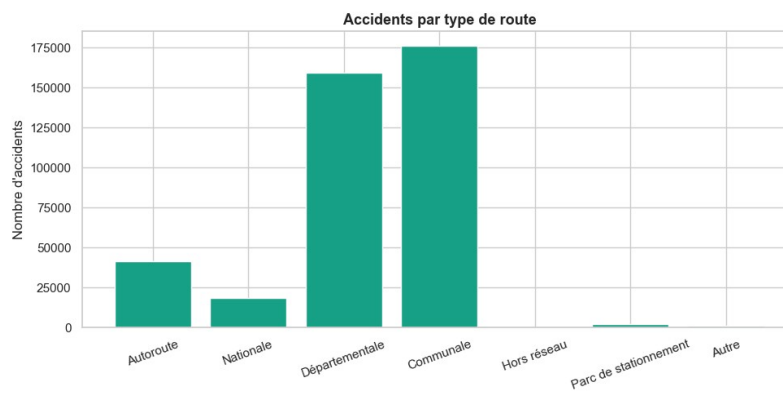
```

	Num_Acc	id_vehicule	num_veh	senc	catv	obs	obsm	choc	manv	motor	occute
0	202200000001	813 952	A01	1	2	0	2	1	9	1	NaN
1	202200000001	813 953	B01	1	7	0	2	2	1	1	NaN

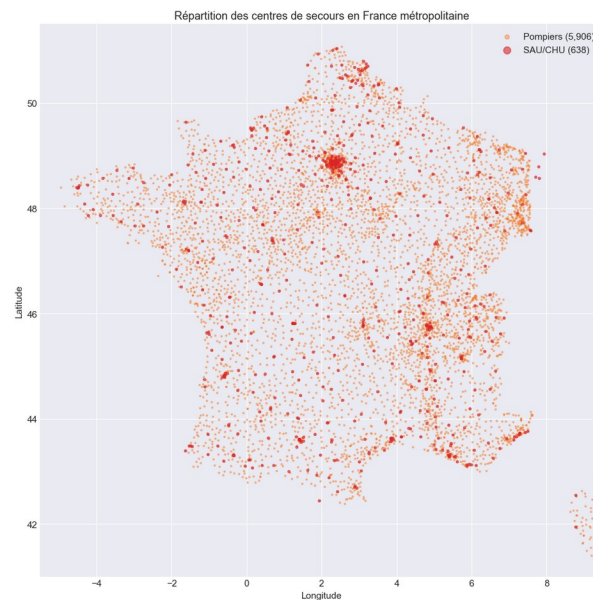
A7 — Aperçu des tables usagers et vehicules



A8 — Nombre d'accidents par année après nettoyage



A9 — Répartition des accidents par catégorie de route (notebook_01)



A10 — Casernes (CIS) et SAU utilisés pour le calcul OSRM (notebook_04)